Retrieval-Assisted Instantiation of Natural-Language Optimization Problems

Soroush Vahidi[1*]

[1*]Department of Computer Science, Ying Wu College of Computing, New Jersey Institute of Technology, University Heights, Newark, 07102-1982, NJ, USA.

Corresponding author(s). E-mail(s): sv96@njit.edu (ORCID: 0000-0003-1934-6282);

Abstract

Purpose: Many optimization problems are first described in natural language before being formalized as mathematical models. This paper studies retrieval-assisted optimization-problem instantiation: retrieving a compatible schema from a fixed catalog and deterministically grounding its scalar parameters from text, in order to separate schema-identification difficulty from downstream scalar-grounding difficulty. **Methods:** We evaluate classical lexical retrieval (TF–IDF, BM25, LSA) and a pretrained dense retriever (BGE-M3) for schema identification on the NLP4LP benchmark (331 queries, a 335-candidate schema catalog), paired with a fully deterministic, inference-time-LLM-free numeric-extraction and typed-greedy slot-assignment procedure. An oracle schema-selection control isolates downstream grounding difficulty from retrieval error, and paired bootstrap and McNemar tests assess statistical robustness. **Results:** Schema retrieval is already strong (Schema R@1 = **0.9094** for TF–IDF, **0.9456** for BGE-M3). BGE-M3 paired with deterministic grounding reaches the highest InstantiationReady among non-oracle configurations (**0.8248**), while an oracle control with perfect retrieval reaches only **0.8489**, a modest additional gain. An exact residual-error decomposition further shows that even when the correct schema, full slot coverage, and correct coarse types are all achieved, **64.7%** of queries still contain at least one scalar value outside the accepted tolerance, identifying fine-grained value-to-slot correctness – not schema retrieval or coarse type identification – as the dominant remaining bottleneck. **Conclusion:** Fixed-catalog schema retrieval is not the limiting factor in this benchmark; deterministic scalar grounding, and specifically precise value-to-slot correctness, remains the primary open challenge. Restricted structural and solver-backed subset studies suggest

downstream usefulness on compatible cases without establishing benchmark-wide solver readiness or open-domain generalization.

Keywords: natural language processing, optimization modeling, knowledge representation, information retrieval, semantic grounding, intelligent information systems

Acknowledgements. The author is deeply grateful to his mother for her continuous emotional support and thanks his Ph.D. advisor, Professor Ioannis Koutis, for his support, guidance, and encouragement. The author gratefully acknowledges Anders Borum for providing complimentary lifetime access to Secure ShellFish, which was used in preparing this manuscript.

1 Introduction

Many real-world engineering and operations problems are first described in natural language before they are formalized as optimization models. Analysts, planners, and domain experts specify resource limits, costs, capacities, proportions, and operational rules in text long before these elements are translated into a mathematical program. This gap between informal problem descriptions and formal optimization artifacts is a central concern of knowledge engineering: translating unstructured textual knowledge into a structured, machine-interpretable model requires both understanding the natural-language description and aligning it with an appropriate formal structure [1, 23, 24]. A system that reliably connects a textual problem description to an appropriate optimization structure can reduce modeling effort, improve accessibility, and support practitioners during early stages of formalization.

This paper adopts a narrower but practically meaningful formulation: retrieval-assisted optimization-problem instantiation over a fixed catalog of structured optimization schemas. Rather than synthesizing an arbitrary mathematical program, the system first retrieves the most compatible optimization schema from a fixed catalog and then deterministically instantiates that schema's scalar parameters from numeric evidence in the text. This framing separates two issues that end-to-end systems often conflate: identifying the correct broad problem structure, and grounding the numerical content of the query into the correct scalar roles of that structure. Because eligible parameter slots are determined by the predicted schema, downstream performance remains retrieval-dependent by construction, which lets the experiments measure whether improved retrieval translates into better structured recovery. Viewing the catalog as a structured knowledge base and retrieval as knowledge-base selection, the task becomes an instance of knowledge-driven model instantiation, connecting directly to knowledge representation and information retrieval [2, 3, 27].

Full translation from natural language to optimization models remains difficult for both families of existing approaches. Retrieval-based and lexical systems can select plausible candidate structures from text, but they do not by themselves recover the

quantities needed to instantiate a model, and their lexical matching can be brittle under paraphrase. End-to-end large-language-model-based systems can generate formulations, solver code, or executable outputs from text [3, 4, 34]. However, they combine several sources of difficulty at once, making it hard to determine which intermediate capabilities are reliable, which stages contribute most to failure, and which parts of the pipeline provide practical value for engineering workflows [5, 13, 19]. These complementary limitations motivate studying an intermediate capability on its own.

In particular, a question that end-to-end settings obscure is where the main difficulty lies: is it in retrieving the correct broad optimization structure from a known catalog, or in grounding the numerical content of the query into the correct scalar roles once that structure has been identified? Answering this question requires a formulation in which the two stages can be evaluated separately and in which downstream performance remains conditioned on the retrieval decision. Such a decomposition is important not only for diagnostics but also for practice, because it determines where future effort in language-to-optimization pipelines should be invested.

To address this question, we study a transparent pipeline that separates schema selection from downstream parameter instantiation. Given a natural-language query, the system ranks the fixed catalog of candidate schemas using schema retrieval (which can be performed with classical lexical methods or a pretrained dense encoder), selects the top-ranked schema, and uses that schema to determine which scalar parameter slots are eligible. Numeric mentions are extracted from the query by rule-based methods, assigned a coarse type, and matched to eligible slots by a deterministic no-reuse compatibility procedure. The selected schema and all downstream grounding decisions—including eligible slots, extracted mentions, and assignments—are observable and reproducible; the grounding decisions themselves are fully inspectable, providing the transparency required for human oversight in decision-support workflows [7, 22].

We evaluate the framework on the NLP4LP benchmark of natural-language optimization problems [2, 3], a curated collection of linear- and mixed-integer-programming descriptions. Schema retrieval is already strong with classical TF–IDF (Schema R@1 = 0.9094), and modern dense retrieval with BGE-M3 improves it significantly (Schema R@1 = 0.9456). This improvement is accompanied by higher downstream readiness, with the BGE-M3 configuration reaching an *Instantiation-Ready* score of 0.8248. However, an oracle control with perfect schema retrieval reaches 0.8489; this gap, corroborated by a numeric-extraction ablation, suggests that downstream scalar grounding and parameter recovery remain the larger residual challenge. Restricted structural and solver-backed subset studies further suggest downstream optimization-oriented usefulness on compatible cases, without claiming benchmark-wide solver readiness.

To avoid any misunderstanding, we emphasize that the methodological novelty of this work does not lie in the individual algorithms (such as TF–IDF, BGE-M3, or standard rule-based extraction). Rather, the core contribution lies in the retrieval-dependent formulation, schema-conditioned scalar grounding, transparent evaluation decomposition, and the empirical diagnosis of retrieval versus grounding

failure. This paper makes four specific contributions. First, it contributes a *retrieval-conditioned schema-instantiation formulation* in which natural-language optimization understanding is decomposed into two explicit stages, and the retrieved schema determines the downstream eligible scalar slots. Second, it contributes an *inference-time LLM-free deterministic grounding procedure*: a fully deterministic pipeline that maps numeric textual evidence to typed optimization-schema parameters through rule-based numeric extraction, coarse type inference, and no-reuse compatibility-based slot assignment, with every intermediate decision inspectable. Third, it contributes a *diagnostic evaluation* separating retrieval and downstream scalar recovery under lexical, dense, and oracle schema selection; together these consistently identify downstream scalar grounding and parameter recovery—not schema retrieval—as the larger residual challenge. Fourth, it contributes *reproducibility and downstream validation*: benchmark, robustness, structural, solver-backed, and external component-level evidence with explicitly scoped claims.

The remainder of the paper is organized as follows. Section 2 reviews related work. Section 3 presents the problem formulation, retrieval stage, deterministic parameter-instantiation backbone, and evaluation-oriented design choices. Section 4 describes the experimental setup and reports retrieval performance, downstream utility, structural and solver-backed validation, and error analysis. Section 5 concludes with the main findings, limitations, and directions for future work.

2 Related Work

Recent advances in natural-language-to-optimization modeling reflect a shift from entity extraction to end-to-end generative systems, and increasingly to structured grounding and retrieval. We organize this literature around four primary axes that situate the present study.

2.1 End-to-End LLM Optimization-Model Generation

An increasingly prominent direction uses large language models (LLMs) to produce complete optimization formulations, solver code, or executable models from natural-language descriptions. Early benchmark efforts such as NL4Opt focused on entity extraction and formulation generation [24], while systems like OptiMUS [3] and OptLLM [34] demonstrated that LLMs could generate and iteratively debug linear and mixed-integer programming models. Recent work has increasingly focused on fine-tuning and synthetic data generation. Systems such as LLMOPT [14] fine-tune models on structured optimization formulations, and ORLM develops a customizable training framework with solver-guided feedback [13]. To enrich training corpora, Opt-MATH introduces a scalable bidirectional data synthesis framework [19]. For more advanced training, Solver-Informed Reinforcement Learning (SIRL) employs verifiable solver feedback to train LLMs via reinforcement learning [9], and OR-R1 automates modeling via test-time reinforcement learning [11]. The DeepOR architecture further proposes a deep-reasoning foundation model specifically for optimization modeling [29].

2.2 Retrieval, Memory, and Solver-Feedback Systems

To improve the correctness of generated models, recent systems increasingly incorporate retrieval, memory, and solver feedback into the generative process. OPT2CODE combines retrieval and code generation to translate linear programs into solver-executable formulations [4], while AUTOFORMULATION maps the problem to a search over hierarchically decomposed components using Monte-Carlo tree search [5]. Extending test-time search, Yu et al. explicitly decompose the generation process into instruction, assumptions, formulation, and solver-code stages using an uncertainty-aware search mechanism [31]. Under multi-agent designs, CHAIN-OF-EXPERTS coordinates role-specialized agents [28], and Zhang et al. propose a dual-cluster memory agent to separate modeling and coding knowledge for structured retrieval of historical solutions [35]. Furthermore, experience replay mechanisms have been introduced to maintain persistent correction memory from solver tracebacks [30]. These systems use retrieval primarily to augment generative LLM pipelines; they generally target open-ended formulation, extending to broader tasks such as solver-independent automated problem formulation [17] and question partitioning [20].

2.3 Binding and Structured Grounding

As benchmarks mature, researchers have exposed a steep performance drop when transitioning from simple puzzles to complex formulations [18]. This drop has motivated a critical distinction between a system's *modeling* ability (selecting mathematical structure) and its *binding* ability (grounding specific numeric parameters and data from text into that structure). Recent work by Gao et al. explicitly identifies data binding as a major limitation of generative text-to-optimization systems, reporting an effective binding limit in open-ended formulations [12]. Earlier component-level approaches such as NER4OPT [15] framed optimization modeling as an information-extraction problem over natural-language entities and quantities. In the broader data and knowledge engineering (DKE) literature, structured grounding and schema matching remain central challenges [1, 27, 32]. These principles are increasingly relevant for optimization tasks, such as automated equivalence checking [33], and parallel broader trends in semantic query answering and conceptual design [6, 25].

Our study addresses a narrower but complementary setting to the binding challenges identified in recent work [12]. Rather than generating an open-ended optimization formulation and subsequently binding its data, we retrieve one schema from a fixed catalog and study how retrieval errors alter the scalar slot inventory available to a deterministic grounding stage.

2.4 Position of the Present Work

The present work isolates an empirically important question often obscured in end-to-end generative pipelines: whether the main difficulty in a fixed-catalog setting lies in retrieving the correct broad optimization structure or in grounding the numerical content of the query into the correct scalar roles once that structure has been identified. To study this, we make downstream slot eligibility depend explicitly on the retrieved schema, structurally tying downstream evaluation to schema-selection quality.

Unlike most recent end-to-end text-to-optimization systems, the proposed grounding procedure does not require generative LLM inference. The scalar grounding procedure is fully deterministic. The grounding stage remains fully inspectable because numeric extraction, typing, and slot assignment follow deterministic rules. Retrieval outputs are reproducible under the fixed lexical or BGE-M3 configurations, but the dense encoder itself is not a rule-based interpretable component. By avoiding generative LLM inference for grounding, the system still preserves intermediate-stage traceability across both configurations. This is not a claim that deterministic grounding is universally superior to LLM-based generative modeling; the two approaches address different points on a cost–capability tradeoff. LLM systems target broader, open-ended synthesis, whereas our pipeline targets fixed-catalog schema instantiation. We therefore position this paper as a study of retrieval-assisted optimization knowledge instantiation: a deterministic, inference-time LLM-free grounding procedure that isolates and measures schema retrieval and schema-conditioned scalar instantiation as distinct components.

3 Methodology

3.1 Problem Formulation

We study a retrieval-assisted instantiation task for natural-language optimization descriptions. Given a query that describes an optimization problem, the objective is not to generate a complete mathematical program directly. Instead, the system produces two structured outputs: (i) a schema prediction that identifies the most compatible optimization template from a fixed catalog, and (ii) scalar parameter assignments for the retrieved schema. This formulation captures an important intermediate step in natural-language-to-optimization pipelines: before a full model can be constructed, the system must identify an appropriate template and recover the key quantities needed to instantiate it [2, 3, 24].

We cast the task as a two-stage decision process. In the first stage, the system performs *schema retrieval*: given a query q, it ranks a catalog $\mathcal{S} = \{s_1, \ldots, s_M\}$ of candidate optimization schemas and returns the top-ranked schema

$$\hat{s} = \arg\max_{s \in \mathcal{S}} \text{score}(q, s). \tag{1}$$

Each schema represents an optimization template together with its associated parameter structure. In the second stage, the system performs *parameter instantiation*: conditioned on the retrieved schema $\hat{s}$, it extracts numeric mentions from the query and assigns them to the scalar parameter slots specified by $\hat{s}$ using deterministic grounding rules. The second stage is explicitly retrieval-dependent: if schema retrieval is incorrect, the set of eligible parameter slots may also be incorrect, reducing downstream coverage and readiness even when the query still contains informative numeric evidence.

In this paper, the term *schema* refers to the retrieved optimization template together with the scalar parameter structure to be instantiated. The resulting task

is narrower than full natural-language-to-optimization translation. The main benchmarked setting does not attempt to synthesize a complete optimization model, recover all variables and constraints, or reconstruct structured parameter objects such as lists, vectors, or dictionaries. Rather, the focus is on whether schema retrieval can anchor a natural-language description to a plausible optimization template, and whether deterministic grounding can recover useful scalar values once that template has been selected.

Our scope is intentionally restricted in three ways. First, benchmark-level downstream evaluation is limited to scalar numeric parameters. Second, the grounding stage itself is fully deterministic: numeric extraction, coarse typing, and slot assignment use fixed rule-based procedures, require no generative LLM or learned extractor, and use no model fine-tuning. Schema retrieval is evaluated both with classical lexical methods and with the off-the-shelf pretrained BGE-M3 dense encoder. Third, the main benchmark study evaluates schema-conditioned scalar recovery rather than full end-to-end model generation. These restrictions are deliberate: they isolate an interpretable intermediate capability and allow precise analysis of where performance gains and failures arise. At the same time, the broader experimental section complements this benchmarked formulation with structural and solver-backed subset validation, including structural-validity analysis and a restricted solver-backed study on compatible instances. These additional experiments are intended to assess downstream practical relevance, not to redefine the core task studied here.

Formally, let $K_G(q)$ denote the set of scalar gold parameter keys associated with query q, and let $P(\hat{s})$ denote the set of scalar parameter keys exposed by the retrieved schema $\hat{s}$. The downstream evaluator considers only the eligible slot set

$$E(q, \hat{s}) = K_G(q) \cap P(\hat{s}), \tag{2}$$

that is, the scalar gold parameters whose keys are present in the predicted schema. To mathematically represent gold values, we define a mapping $y_q : K_G(q) \to \mathbb{R}$ that associates each gold parameter key with its ground-truth scalar value, and let $\hat{y}_q : A(q) \to \mathbb{R}$ represent the predicted values for the subset of assigned slots $A(q) \subseteq E(q, \hat{s})$. This choice makes downstream evaluation retrieval-dependent by construction. Even when a query contains the correct numeric evidence, an incorrect schema can reduce structural overlap and therefore lower downstream performance. The formulation is designed to measure not only whether the correct schema is retrieved, but also whether improved schema retrieval yields measurable gains in structured parameter instantiation.

Retrieval quality is measured by whether the top-ranked schema matches the gold schema, while downstream quality is measured by how well scalar slots from the retrieved schema can be instantiated from the query text. As the experiments later show, retrieval on NLP4LP is already strong, while the larger residual challenge lies in downstream scalar grounding and parameter recovery, including extraction, coarse typing, and slot assignment. The formulation adopted here therefore separates and measures the two main phases: schema retrieval and the downstream scalar-recovery stage as a whole.

3.2 Retrieval-Based Schema Identification

The first stage of the framework performs *schema identification by retrieval*. Given a natural-language optimization description, the system selects the most compatible candidate from a fixed catalog of benchmark-induced schema documents. We formulate this stage as a ranking problem over a finite candidate set $\mathcal{S}$. Each candidate document is indexed by a schema identifier and paired with the textual schema description used by the evaluation pipeline. Given an input query, the retriever scores the query against every candidate document and returns a ranked list. In the present pipeline, only the highest-ranked candidate is used, so retrieval is treated as a top-1 schema selection problem rather than a shortlist-generation or interactive retrieval task.

It is important to clarify what a "schema candidate" means in this setting. The retrieval corpus is the benchmark-induced catalog used by the evaluation pipeline, which contains 335 candidate schema documents. Thus, the retrieval task in this paper is neither open-domain optimization search nor coarse classification into a small number of broad problem families. Instead, it is a fixed-catalog identification problem: for each query, the system must select the single benchmark schema document that serves as the structural template for downstream grounding. This benchmark-specific formulation is deliberate because the goal of the paper is to study whether natural-language optimization descriptions can first be anchored to a compatible template and then instantiated deterministically.

Operationally, the retrieved schema serves as the structural anchor for downstream instantiation. Through its identifier, the system accesses the associated parameter structure and uses that structure to determine which scalar slots are eligible for filling. The retrieval stage therefore does *not* attempt to reconstruct a full symbolic optimization model, derive solver-ready constraints, or generate variables and objectives directly. Instead, it selects the most plausible template index from which the downstream stage can obtain an expected set of scalar parameter roles [2, 3, 24].

This fixed-catalog setup should also be distinguished from label-only classification. The retriever does not predict an abstract class name from a small closed set of manually engineered categories. Rather, it ranks textual schema documents using their natural-language content. This distinction matters because the empirical question in our benchmark is not merely whether a query belongs to a broad optimization family, but whether it can be aligned to the *specific* schema document whose parameter inventory is then used for downstream slot grounding. Retrieval quality therefore directly affects the structural conditions under which the second stage operates.

We evaluate three classical, deterministic lexical retrieval baselines—BM25 [26], TF–IDF cosine retrieval [21], and latent semantic analysis (LSA) [10]—and one modern off-the-shelf dense retriever, BGE-M3 [8]. The classical baselines operate directly on schema-catalog text without benchmark-specific training, dense neural encoders, or learned reranking, and are fully CPU-compatible. In contrast, BGE-M3 utilizes a pretrained dense neural text encoder. Table 1 summarizes all retrieval methods evaluated in our pipeline.

Table 1 Retrieval methods used for schema identification.

Method	Representation	Role in pipeline
BM25	Probabilistic lexical weighting	Sparse lexical baseline
TF–IDF	Weighted bag-of-words cosine similarity	Primary lexical baseline
LSA	Latent projection of TF–IDF features	Lower-dimensional lexical-semantic baseline
BGE-M3	Pretrained dense text embeddings	Modern semantic retrieval baseline

For a query q, each retriever produces a score for every candidate schema $s \in \mathcal{S}$, and the predicted schema is

$$\hat{s}(q) = \arg\max_{s \in \mathcal{S}} \text{score}(q, s). \tag{3}$$

Only $\hat{s}(q)$ is passed to the downstream instantiation stage. This top-1 design matches the evaluation goal of the paper: we ask whether the system can identify a single compatible schema that supports deterministic slot filling, rather than whether it can produce a longer candidate list for later interaction or reranking.

To interpret retrieval performance, we use two diagnostic references in the broader pipeline. The first is an *oracle* control, which replaces the retrieval output with the gold schema identifier and therefore exposes downstream behavior under perfect schema selection. The second is a *random* reference. For retrieval-only evaluation, this random reference is the theoretical top-1 chance rate under uniform selection over the 335 candidate schema documents, namely $1/335 \approx 0.0030$. For downstream evaluation, "random" instead denotes a uniformly chosen schema passed through the same deterministic instantiation pipeline. These references are included only to calibrate interpretation; they are not competing learned retrievers.

Because the retrieved schema determines the candidate slot inventory used downstream, retrieval errors propagate structurally. If the wrong schema is selected, the second stage may inherit a slot set that only partially overlaps the gold instance or that mismatches the intended scalar roles altogether. This can reduce downstream coverage, type agreement, and per-query readiness even when the query still contains relevant numeric evidence. Conversely, strong retrieval creates more favorable structural conditions for deterministic grounding by supplying a more appropriate set of candidate slots.

This formulation emphasizes retrieval-assisted schema grounding rather than full optimization-model reconstruction: the first stage maps a free-form optimization description to a single candidate template whose scalar parameter structure can then be instantiated transparently. Retrieval is an important enabling stage, while the experiments show that substantial residual challenge remains in downstream scalar grounding and parameter recovery.

3.3 Deterministic Parameter Instantiation

Given a natural-language query and a predicted schema, the second stage of the framework instantiates scalar parameter slots from textual numeric evidence. The output of this stage is a set of slot–value assignments associated with the retrieved schema. In the main benchmarked setting, this stage does not generate a complete optimization model and does not recover variables, constraints, or structured parameter objects. Its purpose is narrower: to recover scalar values that can be aligned with the parameter structure of the retrieved template.

The instantiation stage begins by extracting candidate numeric mentions from the query text using a fixed rule-based pattern matcher. The extractor recognizes integers, decimals, percentage expressions, currency-like forms, and comma-separated magnitudes. In degraded query settings, masked numeric tokens are preserved but treated as numerically unknown, so they cannot support exact value recovery. Each extracted mention is then assigned a coarse type label from four buckets: *percent*, *integer*, *currency*, and *float*. Mention typing relies on deterministic surface cues such as percentage markers, currency markers, integer-valued forms, and local lexical context.

The predicted schema determines which parameter slots are eligible for instantiation. Operationally, the system obtains the parameter keys associated with the retrieved template and then restricts attention to scalar-valued slots. This restriction is deliberate. List-valued and dictionary-valued parameters correspond to more structured objects whose recovery would require additional representational assumptions, so the present framework focuses on scalar numeric instantiation. The instantiation problem is therefore defined over a finite set of eligible scalar slots together with a pool of extracted numeric candidates.

For each eligible slot, an expected type is inferred from its parameter name using deterministic rules. Parameters whose names indicate rates or fractions are mapped to *percent*; names indicating counts or cardinalities are mapped to *integer*; names suggesting monetary quantities are mapped to *currency*; and all remaining scalar slots are mapped to *float*. This shared typed representation places slots and extracted numeric mentions in a common compatibility space before assignment.

The simplest assignment rule used in the framework is typed greedy matching. Let $\mathcal{P} = \{p_1, \ldots, p_m\}$ denote the eligible scalar slots induced by the predicted schema, and let $\mathcal{T} = \{t_1, \ldots, t_n\}$ denote the extracted numeric mentions. In the typed-greedy setting, the selected mention for slot p_i is the highest-scoring unused candidate under a deterministic compatibility rule,

$$t_i^* = \arg \max_{t \in \mathcal{T} \setminus \mathcal{U}_{i-1}} \text{compat}(p_i, t), \tag{4}$$

where $\mathcal{U}_{i-1}$ is the set of mentions already assigned in previous steps. The compatibility function prioritizes agreement between the expected slot type and the inferred mention type, together with deterministic tie-breaking based on local cues and extraction order. Once a mention is assigned, it is removed from the candidate pool and cannot be reused for later slots. This yields a one-pass deterministic assignment rule rather than a global combinatorial optimizer.

More sophisticated assignment strategies could be explored, but the present study evaluates the typed-greedy procedure described above. It serves as a representative and fully deterministic baseline that directly exposes the core challenges of type compatibility and local tie-breaking. In exploratory repository-documented experiments (not part of the main benchmark reported here because they were evaluated under an earlier extraction configuration and are therefore not directly comparable to the frozen results above), a global exact bipartite max-weight-matching assignment over the same local compatibility score, and a selective top-k schema-and-grounding reranking variant, were both found not to improve over typed-greedy once schema-gated correctly, and the reranking variant's apparent gain under the ordinary *Instantiation-Ready* metric was traced to a wrong-schema artifact rather than a genuine grounding improvement (Section 4.5). These negative findings are documented in the accompanying repository and motivate treating typed-greedy as a clear, unconfounded baseline rather than as a placeholder awaiting a better assignment rule. Regardless of the specific assignment heuristic, any such rule-based method shares the same underlying structure: deterministic numeric extraction, schema-conditioned scalar slot selection, coarse type inference, and no-reuse slot filling. This common backbone is summarized in Algorithm 1 and illustrated in Figure 1.

Optional deterministic, lexicon-based tie-breaking cues can also be defined. For instance, slot names and local query contexts can be matched against a fixed lexicon of optimization-role cue words (e.g., terms indicating limits, cost coefficients, or cardinality bounds) to break ties among type-compatible candidates. While such heuristic role-matching rules can help resolve local ambiguities, they are not part of the primary configurations evaluated in our main benchmark, and they provide no formal feasibility or correctness guarantees.

To isolate the effect of the ratio-aware numeric-extraction refinement, we also evaluate a *numeric-extraction ablation* that corrects multiplicative ratio-word numeric expressions (e.g., "twice", "double", "triple") so that extracted values are scaled deterministically. As the experiments later show, this correction materially improves *TypeMatch* and *InstantiationReady* while leaving retrieval-dependent quantities much less changed, confirming that improved handling of multiplicative ratio expressions can increase downstream readiness without altering schema retrieval. We explain the correction only once here and refer to the comparison throughout as the numeric-extraction ablation.

In short, the instantiation stage is intentionally simple: numeric evidence is extracted from the query, typed using deterministic cues, filtered through the scalar parameter structure of the predicted schema, and assigned to slots by a deterministic no-reuse procedure. The resulting slot–value mapping is then evaluated downstream. In the broader paper, this benchmarked scalar-instantiation stage is complemented by restricted downstream validation on subsets, but the core methodology here remains a deterministic, schema-conditioned scalar grounding pipeline.

3.4 Evaluation-Oriented Design Choices

The methodology is intentionally designed so that downstream evaluation remains sensitive to schema retrieval quality. In particular, the set of parameter slots eligible

Algorithm 1 Deterministic parameter instantiation pipeline. Numeric mentions are extracted from the query and typed using surface cues. In parallel, the predicted schema provides the eligible scalar slots, whose expected types are inferred from slot names. A deterministic compatibility-based assignment procedure then maps numeric mentions to slots without token reuse using a deterministic compatibility rule. Alternative deterministic compatibility rules can be layered on this backbone.

Require: Query text q, predicted schema $\hat{s}$
Ensure: Slot–value mapping A
Extract numeric mentions $\mathcal{T}$ from q
Infer a coarse type for each mention in $\mathcal{T}$
Obtain parameter keys from schema $\hat{s}$
Restrict to eligible scalar slots $\mathcal{P} = \{p_1, \ldots, p_m\}$
Infer an expected type for each slot in $\mathcal{P}$
Initialize assignments $A \leftarrow \emptyset$ and used-mention set $\mathcal{U} \leftarrow \emptyset$
for each slot $p_i \in \mathcal{P}$ **do**
    Define candidate set $\mathcal{C}_i \leftarrow \mathcal{T} \setminus \mathcal{U}$
    if $\mathcal{C}_i = \emptyset$ **then**
        continue
    end if
    Score mentions in $\mathcal{C}_i$ using a deterministic slot–mention compatibility rule
    Select the highest-ranked mention t_i^*
    Assign $A[p_i] \leftarrow \text{value}(t_i^*)$
    Update $\mathcal{U} \leftarrow \mathcal{U} \cup \{t_i^*\}$
end for
return A

for instantiation is determined from the *predicted* schema rather than from the gold schema. This choice is central to the formulation. If eligible slots were defined directly from the gold record, downstream coverage and readiness would no longer reflect schema-selection quality, and the evaluation would partially decouple retrieval from instantiation. By conditioning slot eligibility on the retrieved template, the framework ensures that retrieval errors propagate naturally into downstream performance through reduced structural overlap, lower slot availability, and weaker query-level readiness. At the same time, the evaluation avoids leakage: gold information is used for scoring, not for determining which schema is instantiated at inference time.

A second design choice restricts the main benchmarked downstream evaluation to scalar numeric parameters, which provide a common and reproducible substrate for comparing retrieval-conditioned instantiation across heterogeneous templates while avoiding the additional alignment and dimensionality assumptions required to reconstruct list- or dictionary-valued objects (Section 3.1).

As introduced in Section 3.2, the oracle and random references serve the same interpretive purpose throughout the paper. We refer to the oracle condition as an *oracle control* (or oracle reference) rather than a formal upper bound: because *InstantiationReady* depends on per-query type-inference and slot-eligibility details that are not guaranteed to be monotonic in schema correctness, the oracle condition is not

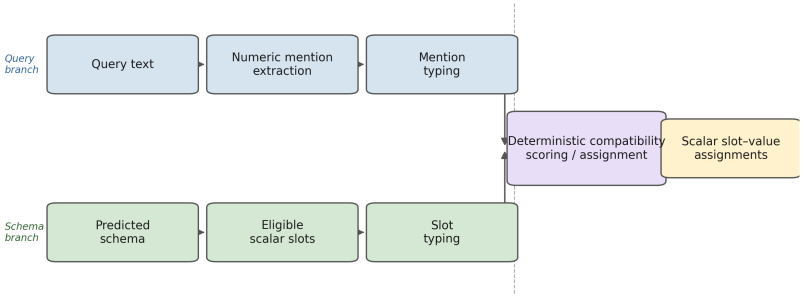

Fig. 1 Schema-conditioned deterministic parameter instantiation. Numeric mentions are extracted from the query, typed using surface cues, filtered through the scalar slot inventory of the predicted schema, and assigned by a no-reuse compatibility rule. Alternative deterministic compatibility rules can be layered on this backbone.

mathematically certain to dominate every non-oracle method on every metric or subset, and Section 4.7 reports a concrete case (the 20-instance solver-backed subset) where it does not.

The core benchmark metrics are solver-free, and the downstream grounding stage is deterministic: no feasibility test, optimization run, or objective-level verification enters the core metrics, and the grounding procedure uses no generative LLM or learned extractor. Schema retrieval is evaluated separately using both classical lexical baselines and the pretrained BGE-M3 dense encoder. This keeps the central evaluation transparent and reproducible and lets the experiments answer a focused question – how much structured instantiation utility schema retrieval and deterministic rule-based grounding can provide – while downstream executability is examined separately on the restricted subsets of Section 4.7.

These choices also determine how the metrics should be read. *InstantiationReady* is a diagnostic readiness criterion based on coverage and type consistency, not a guarantee of model correctness or solver compatibility; strong schema retrieval is not full optimization-model recovery; and successful scalar assignment is not feasibility- or optimality-verified instantiation. The framework is best viewed as a retrieval-assisted deterministic instantiation system that quantifies how schema grounding affects structured parameter recovery, with the restricted subset analyses addressing the separate question of downstream executable behavior on compatible cases.

4 Experiments

4.1 Experimental Setup

We evaluate the proposed retrieval-assisted instantiation pipeline on the NLP4LP benchmark [2, 3], a curated collection of linear- and mixed-integer-programming problem descriptions in which each instance pairs a natural-language description with a

gold parameter record, a reference solver implementation, and, where available, a reference solution. Unless otherwise stated, the main benchmark results are measured on the benchmark test split, which contains 331 query instances. Each test query has exactly one gold optimization schema. Each test query is ranked against the benchmark-induced schema catalog used by the evaluation pipeline, which contains 335 candidate schema documents. Accordingly, retrieval is a 335-way top-1 identification task, and the corresponding random top-1 reference is the theoretical chance rate under uniform selection over this catalog, namely $1/335 \approx 0.0030$.

To assess robustness to surface degradation and reduced context, we evaluate three benchmark query variants. The *orig* (original) variant uses the original problem descriptions. The *noisy* variant applies controlled lexical corruption, including lowercasing, token deletion, stopword reduction, and masking of some numeric expressions. The *short* variant retains only the first sentence of each description. These variants allow us to test the same retrieval-and-grounding pipeline under full context, degraded wording, and substantially reduced context.

The retrieval stage ranks candidate schemas using only schema-catalog text. In the final reported benchmark comparison, we evaluate three classical retrieval baselines—BM25 [26], TF–IDF cosine retrieval implemented with SCIKIT-LEARN [21], and latent semantic analysis (LSA), obtained by applying truncated singular value decomposition to TF–IDF vectors [10]—and one modern off-the-shelf dense retriever, BGE-M3 [8]. The classical baselines (BM25, TF–IDF, and LSA) operate as deterministic, lexical-matching retrievers, while BGE-M3 is a pretrained dense neural text encoder used off-the-shelf without benchmark-specific fine-tuning. In all downstream experiments, only the top-ranked schema is passed to the instantiation stage. We also report an *oracle* control in which the retrieved schema is replaced by the gold schema, thereby isolating downstream grounding difficulty from retrieval error.

Given a query and a predicted schema, the downstream stage instantiates scalar parameter slots deterministically from the query text. Numeric mentions are extracted using a fixed rule-based pattern matcher. In the *noisy* variant, masked numeric tokens are preserved but treated as numerically unknown; they may therefore still indicate the presence of a quantity without supporting exact value recovery. We restrict the core benchmarked downstream evaluation to scalar numeric parameters and exclude list-valued and dictionary-valued parameters so that the benchmark targets scalar slot grounding rather than structured arrays, matrices, or other higher-order objects.

The downstream candidate slot set is derived from the *predicted* schema rather than the gold schema. When schema metadata explicitly provides parameter keys, we use those keys; otherwise, we extract keys from the predicted parameter record. These predicted keys are then intersected with scalar gold parameters to determine the subset of slots eligible for evaluation. This makes downstream evaluation intentionally retrieval-dependent: if the wrong schema is retrieved, the candidate slot inventory may already be structurally misaligned with the gold instance, which can reduce both attainable coverage and value accuracy even when the query still contains relevant numbers.

To assign extracted numbers to slots, we evaluate the deterministic grounding backbone described in Section 3.3. Across all methods, scalar slots are typed

using four lightweight rule-based categories: *percent*, *integer*, *currency*, and *float*. Extracted numeric mentions are mapped into the same categories using surface cues such as percentage markers, currency symbols, and integer-valued forms. The evaluated benchmark reported in this paper explicitly includes four configurations: (i) the principal TF–IDF typed-greedy baseline with baseline extraction; (ii) TF–IDF typed-greedy with ratio-aware extraction (designed to isolate the effect of the ratio-aware numeric-extraction ablation); (iii) BGE-M3 retrieval paired with the same ratio-aware deterministic grounding stage (designed to test whether a modern semantics-aware retriever improves downstream utility while holding the grounding procedure fixed); and (iv) an oracle typed-greedy control (perfect schema selection paired with typed-greedy grounding). Crucially, the downstream grounding procedure remains identical when comparing the TF–IDF and BGE-M3 ratio-aware configurations; BGE-M3 modifies the schema-retrieval stage only, and no learned or large-language-model-based grounding is introduced. Alternative deterministic assignment heuristics are possible, but the primary comparison reported here uses typed-greedy grounding to provide a clear, unconfounded baseline for the core retrieval and grounding stages.

We report both retrieval effectiveness and downstream instantiation quality. *Schema R@1* is the fraction of test queries whose top-ranked schema matches the gold schema. For downstream evaluation, let G_i denote the scalar gold slots for query i that are eligible for evaluation after intersecting gold scalar parameters with the candidate slots induced by the predicted schema, and let $A_i \subseteq G_i$ denote the subset of those slots that receive an assignment. Then *Coverage* is

$$\mathit{Coverage} = \frac{\sum_i |A_i|}{\sum_i |G_i|}, \tag{5}$$

that is, the fraction of eligible scalar gold slots that receive any assignment. *TypeMatch* is the fraction of assigned slots whose predicted numeric type matches the expected slot type,

$$\mathit{TypeMatch} = \frac{\sum_i |\{s \in A_i : \tau_{\text{pred}}(i,s) = \tau_{\text{slot}}(s)\}|}{\sum_i |A_i|}, \tag{6}$$

where $\tau_{\text{pred}}(i,s)$ denotes the coarse type category of the extracted numeric mention assigned to slot s of query i, and $\tau_{\text{slot}}(s)$ is the expected coarse type inferred from the parameter slot name.

To measure value accuracy, we report *Exact20 (on hits)*. This metric is computed only on the subset of *schema-hit* queries for which a comparable numeric prediction and gold value are both available for the slot under consideration. To handle zero and near-zero gold values robustly, relative error is calculated using a bounded relative error formula:

$$\text{error}(i,s) = \frac{|\hat{y}_{i,s} - y_{i,s}|}{\max(1.0, |y_{i,s}|)}, \tag{7}$$

where $\hat{y}_{i,s}$ and $y_{i,s}$ denote the predicted and gold numeric values for slot s of query i. A slot assignment is counted as correct if $\text{error}(i,s) \leq 0.20$. Because meaningful numeric comparison requires both schema correctness and a comparable numeric pair, *Exact20 (on hits)* is intentionally normalized on this eligible subset rather than on

all test queries. It should therefore be interpreted as a conditional value-grounding measure rather than as an end-to-end benchmark score.

Our main downstream readiness metric is *InstantiationReady*. For query i, let $Coverage_i$ and $TypeMatch_i$ denote the per-query coverage and type-match rates computed over the predicted-schema-conditioned eligible slot set $E(q_i, \hat{s}_i)$ (Eq. (2)). Specifically, if $A_i \subseteq E(q_i, \hat{s}_i)$ denotes the set of slots receiving assignments, we define the per-query coverage as $Coverage_i = |A_i|/|E(q_i, \hat{s}_i)|$ when $|E(q_i, \hat{s}_i)| > 0$, and $Coverage_i = 0$ when $E(q_i, \hat{s}_i) = \emptyset$. Similarly, the per-query type-match rate is $TypeMatch_i = |\{s \in A_i : \tau_{\text{pred}}(i, s) = \tau_{\text{slot}}(s)\}|/|A_i|$ when $|A_i| > 0$, and $TypeMatch_i = 0$ when $A_i = \emptyset$. A query is counted as *InstantiationReady* if both rates reach a fixed, diagnostic readiness threshold,

$$InstantiationReady_i = \mathbb{1}[Coverage_i \geq 0.8 \ \wedge \ TypeMatch_i \geq 0.8], \tag{8}$$

and the reported *InstantiationReady* score is the mean of $InstantiationReady_i$ over the benchmark denominator. This 0.8/0.8 threshold is a fixed diagnostic readiness criterion rather than a theoretically optimal cutoff; it is designed to represent a high-fidelity recovery of the necessary parameter interface. Because $E(q_i, \hat{s}_i)$ is itself schema-dependent, an incorrect schema prediction typically depresses $Coverage_i$ and lowers the chance of reaching the threshold, but exact schema match is not enforced as a separate hard gate in this rule. Thus, *InstantiationReady* is a per-instance readiness-threshold criterion that combines schema-conditioned slot coverage and type compatibility into a single diagnostic, rather than an all-or-nothing full-coverage criterion.

Unless otherwise stated, the retrieval metrics and the main downstream readiness metrics are reported over the full benchmark denominator for each variant. The only exception is *Exact20 (on hits)*, which is explicitly conditional on the subset of schema-hit cases with comparable numeric values. We additionally report bootstrap confidence intervals and selected paired significance analyses for key retrieval and downstream method pairs, and overlap-based stress tests – including stratified retrieval performance by lexical-overlap level and retrieval ablations under sanitized text conditions such as number removal and stopword stripping – to test whether strong retrieval may partly reflect benchmark-specific lexical overlap (Section 4.5).

To isolate the effect of corrected numeric extraction, we also report a numeric-extraction ablation comparing the baseline and ratio-aware configurations of the TF–IDF typed-greedy method on the original queries, together with a strict schema-gated robustness check (Section 4.4). Both the main benchmark and the ablation are measured on the canonical NLP4LP test benchmark.

Finally, to complement the core benchmark with downstream practical evidence, we include three structural and solver-backed validation studies (Section 4.7): (i) a structural validation study (60 instances), (ii) an extended structural validation study (269 instances), and (iii) a restricted solver-backed execution study on a small compatibility-filtered subset (20 instances). These auxiliary studies are not replacements for the main benchmark; rather, they provide complementary structural and solver-backed evidence on restricted subsets. Table 2 summarizes the overall experimental design.

Table 2 Summary of the experimental blocks used in the paper.

Block	Primary purpose	Main outputs
Main benchmark	Retrieval and scalar grounding on NLP4LP	Schema R@1, Coverage, TypeMatch, Exact20 (on hits), InstantiationReady
Robustness analyses	Sensitivity to degraded input and lexical overlap	Orig / noisy / short retrieval comparisons, overlap-based stress tests
Numeric-extraction ablation	Isolate effect of corrected ratio-word extraction	TypeMatch and InstantiationReady changes (baseline vs. ratio-aware extraction)
Significance testing	Assess whether reported gaps are statistically robust	Paired bootstrap p-values and confidence intervals
Structural validation (60 inst.)	Assess downstream structural usefulness	Structural-validity and instantiation-completeness rates
Extended structural validation (269 inst.)	Assess structural behavior on a larger subset with available reference code	Schema hit, structural validity, and instantiation completeness
Restricted solver-backed execution (20 inst.)	Real, restricted solver execution via SciPy HiGHS shim	Executable, solver-success, feasible, and objective-produced rates

The deterministic grounding and classical retrieval experiments are CPU-compatible. The BGE-M3 dense-retrieval baseline uses a pretrained neural encoder and was evaluated with GPU (CUDA) acceleration using float32 precision, L2-normalized 1024-dimensional embeddings, and cosine similarity; no model fine-tuning was performed. The implementation uses standard data-processing and retrieval libraries, including Hugging Face `datasets` [16] and SCIKIT-LEARN [21]. The grounding procedure uses no generative LLM inference, and no model fine-tuning or solver-based scoring enters the core benchmark metrics.

4.2 Schema Retrieval Performance

We first evaluate schema retrieval in isolation to determine whether selecting the correct benchmark schema is itself a major source of error in the present fixed-catalog setting. Table 3 reports *Schema R@1*, defined as the fraction of test queries for which the top-ranked retrieved schema matches the gold schema, across the three query variants described in Section 4.1. Retrieval is evaluated against the benchmark-induced catalog of 335 candidate schema documents. Accordingly, the random reference corresponds to the theoretical top-1 chance rate under uniform selection over this catalog, namely $1/335 \approx 0.0030$.

The purpose of this experiment is deliberately limited. We do not treat it as an open-domain optimization retrieval task or as evidence that lexical retrieval alone

would suffice in broader real-world schema libraries. Rather, within the benchmark-induced candidate set, we ask whether the evaluated retrieval methods can identify the single schema document that provides the downstream scalar slot inventory used by the grounding stage. This benchmark-scoped retrieval analysis is therefore intended to characterize the structural conditions under which downstream instantiation operates, not to claim a general solution to schema retrieval in unrestricted optimization-modeling environments.

Within this fixed-catalog benchmark, schema retrieval is already strong. On the original queries, an off-the-shelf modern dense retriever (BGE-M3 [8]) achieves the best performance with *Schema R@1* = 0.9456, outperforming the strongest lexical baseline (TF–IDF at 0.9094, followed by BM25 at 0.8822 and LSA at 0.8459). This improvement over the TF–IDF baseline is statistically significant under an exact McNemar test ($p = 0.0075$ on discordant pairs, with 15 queries improved by BGE-M3 only versus 3 improved by TF–IDF only). All methods are far above the random baseline, indicating that the benchmark descriptions contain enough schema-level textual evidence for recovering the correct benchmark template in most cases. Thus, in the canonical test setting studied here, schema identification is substantially easier than the downstream instantiation problem studied later in the paper.

Retrieval also remains strong under controlled lexical degradation. On the *noisy* queries, TF–IDF, BM25, and LSA achieve *Schema R@1* values of 0.9033, 0.8912, and 0.8882, respectively, while BGE-M3 achieves 0.9426. These values are close to their corresponding results on the original queries. This stability suggests that benchmark-level schema identification is not highly sensitive to moderate perturbations such as lowercasing, token deletion, stopword reduction, and numeric masking. Under both conditions, BGE-M3 remains the strongest evaluated retriever.

The *short* setting is more challenging. When each description is truncated to its first sentence, *Schema R@1* decreases to 0.7795 for TF–IDF, 0.7674 for BM25, and 0.7644 for LSA, whereas BGE-M3 reaches 0.8157. This drop is expected because truncation removes contextual details that often distinguish closely related benchmark formulations. Even so, BGE-M3 remains the strongest retriever in this degraded setting as well, and performance remains well above chance for all evaluated methods, showing that substantial benchmark-specific schema evidence is often present in the opening sentence alone. This relative robustness to surface degradation characterizes the current fixed-catalog benchmark and does not establish general open-domain robustness.

Across all three variants, BGE-M3 achieves the highest average retrieval accuracy (0.9013). Among lexical methods, TF–IDF achieves the best average accuracy (0.8641), with BM25 close behind (0.8469) and LSA competitive (0.8328). Because TF–IDF requires no model weights and acts as a strong, fully deterministic lexical baseline, we retain it as the main backbone for the primary downstream experiments, but we evaluate BGE-M3 downstream as well to confirm that the residual grounding challenge persists even with a stronger semantics-aware retriever. More importantly, the absolute retrieval levels in Table 3 establish the main benchmark-level interpretive point for the remainder of the paper: within this fixed-catalog NLP4LP setting, downstream scalar grounding and parameter recovery constitute the larger residual

Table 3 Schema retrieval accuracy across query variants, reported as *Schema R@1*. Each value is computed over all 331 test queries in the corresponding variant. The random baseline is the theoretical top-1 chance rate under uniform schema selection over the 335 candidate schemas, i.e., $1/335 \approx 0.0030$. Best result in each column is shown in bold.

Baseline	Orig	Noisy	Short	Avg.
Random	0.0030	0.0030	0.0030	0.0030
LSA	0.8459	0.8882	0.7644	0.8328
BM25	0.8822	0.8912	0.7674	0.8469
TF–IDF	0.9094	0.9033	0.7795	0.8641
BGE-M3 (dense)	**0.9456**	**0.9426**	**0.8157**	**0.9013**

challenge. This conclusion is intentionally benchmark-scoped and should not be read as a claim that schema retrieval is solved in broader optimization-modeling settings.

4.3 Downstream Utility for Problem Instantiation

We next evaluate whether correct or near-correct schema retrieval translates into useful downstream instantiation within the restricted scalar-grounding setting studied in this paper. In the proposed pipeline, the retrieved schema determines the candidate scalar slot inventory, after which deterministic numeric extraction and slot assignment are applied as described in Section 4.1. The central question is not whether the system can recover a complete optimization model, but whether successful schema selection yields scalar slot–value assignments that are sufficiently complete and type-consistent to provide measurable intermediate support for optimization modeling.

Table 4 reports the main downstream results on the original queries, computed from the final evaluation records. Three findings are most important. First, retrieval-assisted grounding with deterministic slot assignment recovers a substantial amount of scalar structure on the original benchmark inputs. Under the principal TF–IDF lexical configuration with ratio-aware extraction, the system achieves *Coverage* = 0.8886, *TypeMatch* = 0.8665, and *InstantiationReady* = 0.8006. The strongest overall non-oracle configuration is BGE-M3 paired with the same ratio-aware deterministic grounding stage, which achieves a further improvement to *Coverage* = 0.9154, *TypeMatch* = 0.8946, and *InstantiationReady* = 0.8248. Under this stronger semantics-aware configuration, the strict gated ready rate reaches *StrictInstantiationReady* = 0.8006, while TF–IDF ratio-aware remains the strongest configuration overall (including the oracle control) specifically on value-grounding accuracy with *Exact20* = 0.2614 (on hits). Within this benchmarked scalar task, these results show that the proposed deterministic grounding pipeline, whether paired with classical lexical or pretrained dense retrieval, can recover a large fraction of eligible scalar assignments with the correct coarse type.

Second, the strict schema-gated criterion (Eq. (9)) slightly reduces the ready rate: the TF–IDF typed-greedy baseline reaches *StrictInstantiationReady* = 0.7704, so a

small number of queries achieve the coverage and type thresholds despite a mismatched predicted schema. This shows that wrong-schema cases have a modest but nonzero effect on the canonical ready score; adding the explicit schema gate reduces the TF–IDF ratio-aware ready rate from 0.8006 to 0.7704 without changing the qualitative conclusion.

Third, the oracle control confirms that retrieval matters but is not the sole remaining challenge within this benchmark. Replacing the TF–IDF schema retriever with BGE-M3 while holding the ratio-aware deterministic grounding stage fixed improves *InstantiationReady* from 0.8006 to 0.8248. Replacing the schema retriever with the oracle control (Oracle-TG) increases it further to 0.8489 (with *StrictInstantiationReady* rising to 0.8489). The inclusion of an off-the-shelf dense retriever does not materially alter the benchmark-level diagnosis: schema identification remains comparatively strong (and dense retrieval improves it further), while the larger residual gap to perfect instantiation arises in schema-conditioned scalar grounding and parameter recovery. We note that while dense retrieval improves overall schema identification and downstream readiness, it does not improve the conditional Exact20 (on hits) metric (0.2436 for BGE-M3 vs. 0.2614 for TF–IDF); indeed, TF–IDF's Exact20 (on hits) is the highest of all four evaluated configurations, exceeding even the oracle control (0.2505). Because Exact20 is conditional on correct schema hits, its denominator and query composition shift with retrieval success – TF–IDF, BGE-M3, and Oracle condition on 291, 303, and 320 comparable schema-hit queries, respectively – so this ordering reflects which *subset* of queries each configuration's hits happen to cover, not a claim that weaker retrieval improves value-grounding accuracy. This non-monotonicity is consistent with a similar oracle-does-not-dominate case reported for the small solver-backed subset in Section 4.7, and it highlights that the remaining downstream recovery errors persist regardless of which schema-selection method is used. Even when the correct schema is supplied, substantial difficulty remains in deciding which extracted quantity should fill which scalar slot. This is the central benchmark-level empirical point of the paper: on NLP4LP, schema retrieval is already comparatively strong, while downstream scalar grounding and parameter recovery remain the larger residual challenge.

Alternative tie-breaking and assignment heuristics can be designed, but we focus on the reported typed-greedy configuration.

Taken together, these results support a narrower but more defensible interpretation of system utility: on original benchmark inputs, the retrieval-assisted pipeline with deterministic grounding produces partial scalar instantiations for a substantial fraction of cases, and supplying the oracle schema does not close the remaining gap to full readiness. These downstream results are meaningful as benchmarked evidence about this restricted intermediate task, not as a claim of full semantic understanding or broad real-world optimization-model recovery.

4.4 Error Analysis and Ablation Studies

To better understand the remaining gap between strong schema retrieval and reliable downstream instantiation, we examine two complementary diagnostics: an exact,

Table 4 Main downstream results on the original queries, computed from the final evaluation records. *Coverage*, *TypeMatch*, *InstantiationReady*, and *StrictInstantiationReady* use the full 331-query denominator. *Exact20* (on hits) is conditional on schema-hit cases with comparable predicted and gold numeric values. Best non-oracle result in each column is shown in bold.

Method	Coverage	TypeMatch	Exact20	InstReady	Strict
TFIDF-TG (baseline extraction)	0.8794	0.8515	0.2449	0.7764	0.7462
TFIDF-TG (ratio-aware extraction)	0.8886	0.8665	**0.2614**	0.8006	0.7704
BGE-M3 (dense) + ratio-aware grounding	**0.9154**	**0.8946**	0.2436	**0.8248**	**0.8006**
Oracle-TG	0.9416	0.9230	0.2505	0.8489	0.8489

Abbreviations: TFIDF-TG = TF–IDF typed-greedy grounding; Oracle-TG = oracle schema selection with typed-greedy grounding; BGE-M3 = pretrained dense text encoder. All values are supported by the accompanying repository artifacts. *Exact20 (on hits)* uses a single, uniformly applied denominator convention across all four rows: the mean of the per-query bounded-error indicator (Eq. (7)) over exactly the schema-hit queries that have a comparable predicted/gold numeric value, excluding (not zero-filling) schema-hit queries with no comparable value. The four rows accordingly condition on 291, 291, 303, and 320 comparable queries, respectively.

reproducible residual-error decomposition of the main benchmark and the numeric-extraction ablation. Together, these analyses clarify where the deterministic pipeline still fails.

A first point is that the residual challenges lie primarily *after* schema retrieval rather than *at* schema retrieval. As shown in Section 4.2, TF–IDF retrieves the correct schema for 0.9094 of the original queries, leaving limited room for large gains from retrieval alone. The decomposition in Table 5 is consistent with this interpretation and quantifies it precisely: wrong-schema retrieval accounts for only 30 of 331 queries (9.1%), while the remaining 301 schema-hit queries split across three further, mutually exclusive outcomes.

Table 5 partitions all 331 queries into five categories, evaluated in a fixed order so that every query falls into exactly one category (no double-counting, unlike a heuristic multi-label taxonomy): *wrong schema* (schema retrieval itself fails); *incomplete coverage* (correct schema, but at least one eligible scalar slot receives no assignment); *type mismatch* (correct schema and full coverage, but at least one assigned value has the wrong coarse type); *value inaccurate* (correct schema, full coverage, and correct types throughout, but at least one comparable scalar value falls outside the 20% bounded relative-error tolerance of Eq. (7)); and *fully correct* (none of the above). This decomposition is computed directly and deterministically from the same frozen per-query evaluation records used throughout this section (reproduction script and output committed alongside the repository's other frozen artifacts), so every count is exactly reproducible rather than a qualitative estimate.

The decomposition reframes the previously reported diagnosis. Type mismatches account for a real but comparatively modest share of outcomes (49 of 331 queries, 14.8%), consistent with the observation that many scalar values in the benchmark are not explicitly marked as percentages, currency amounts, or integers and are therefore more vulnerable to coarse type-inference errors than clearly signaled surface forms. However, the single largest category is *value inaccurate* (214 of 331 queries, 64.7%):

even when the correct schema is retrieved, every eligible slot receives an assignment, and every assigned value has the correct coarse type, the specific extracted numeric value frequently still falls outside the tolerance used by Exact20 for at least one slot. Only 7 queries (2.1%) are fully correct on all four criteria simultaneously. This shows that the dominant remaining bottleneck in this benchmark is not primarily coarse type identification but fine-grained value-to-slot correctness – selecting, among several type-compatible numeric mentions in the query, the one that actually corresponds to the target parameter – which is exactly the ambiguity that the conditional *Exact20 (on hits)* metric (Section 4.1) is designed to expose. Unsupported-schema cases do not occur in this benchmark by construction (every query has a valid gold catalog entry).

Table 6 quantifies the numeric-extraction ablation. On the original queries, correcting multiplicative ratio-word numeric expressions (e.g., "twice", "triple") improves *Coverage* from 0.8794 to 0.8886, *TypeMatch* from 0.8515 to 0.8665, and *InstantiationReady* from 0.7764 to 0.8006, with a corresponding gain in *StrictInstantiationReady* from 0.7462 to 0.7704. The ratio-aware refinement converts 8 additional queries to strict-ready status with no losses, a change that is statistically significant under a McNemar test ($p = 0.008$, Section 4.5). By contrast, retrieval-dependent quantities are unchanged, exactly as expected from the nature of the correction: the system retrieves the same schemas, but now grounds the affected scalar values more accurately.

This ablation is useful for interpreting the downstream results. The gains are real but modest, and they are concentrated in the type-consistency and readiness metrics rather than in retrieval. The remaining gap to the oracle control is therefore not an artifact of the numeric-extraction correction; it reflects genuinely difficult semantic ambiguities – totals versus unit coefficients, lower versus upper bounds, percentages versus absolute values, and generic continuous values whose local lexical context is weak.

In summary, the benchmark results show that template selection is highly accurate, while the larger residual challenge lies in downstream scalar grounding and parameter recovery. The residual-error decomposition indicates that, within that downstream stage, fine-grained value-to-slot accuracy – not coarse type handling – is the dominant remaining contributor.

4.5 Statistical Significance and Lexical-Overlap Robustness

Section 4.1 noted that key retrieval and downstream comparisons are supported by paired bootstrap significance tests and by overlap-based stress tests; we report both here.

Using a paired bootstrap with $B = 10{,}000$ resamples over the 331 original test queries, the *InstantiationReady* gap between the TF–IDF typed-greedy method and the oracle control is statistically significant ($\Delta = -0.0483$, 95% CI $[-0.0755, -0.0242]$, $p < 0.001$), confirming that the oracle improvement discussed in Section 4.3 is unlikely to be a sampling artifact. Under the stricter schema-gated criterion the gap widens and remains significant ($\Delta = -0.0785$, 95% CI $[-0.1088, -0.0514]$, $p < 0.001$; Table 7). The ratio-aware extraction refinement gains 8 strict-ready queries with no losses, which is significant under both the paired bootstrap on the strict metric ($\Delta = +0.0242$, 95% CI $[0.0091, 0.0423]$, $p = 0.0006$) and an exact McNemar

Table 5 Exact, mutually exclusive residual-error decomposition for TF–IDF ratio-aware typed-greedy grounding on the 331 original queries, computed deterministically from the frozen per-query evaluation records. Categories are evaluated in the listed order, so every query is counted exactly once and counts sum to 331.

Category	Count	Fraction
Wrong schema (schema retrieval fails)	30	0.091
Incomplete coverage (correct schema, $\geq$ 1 slot unfilled)	31	0.094
Type mismatch (correct schema, full coverage, $\geq$ 1 wrong type)	49	0.148
Value inaccurate (correct schema, full coverage, correct types, $\geq$ 1 value outside tolerance)	214	0.647
Fully correct (all four criteria satisfied)	7	0.021
Total	331	1.000

Table 6 Numeric-extraction ablation on the original queries, comparing the baseline and ratio-aware extraction configurations of the TF–IDF typed-greedy method. The ratio-aware configuration corrects multiplicative ratio-word numeric expressions. The strict-ready count (of 331) rises from 247 to 255; see Section 4.5 for the significance test.

Metric	Baseline	Ratio-aware	Δ
Coverage	0.8794	0.8886	+0.0092
TypeMatch	0.8515	0.8665	+0.0150
Exact20 (on hits)	0.2449	0.2614	+0.0165
InstantiationReady	0.7764	0.8006	+0.0242
StrictInstantiationReady	0.7462	0.7704	+0.0242

test ($p = 0.008$). The downstream gains from replacing the TF–IDF retriever with BGE-M3 while holding the ratio-aware grounding fixed are also statistically significant: *InstantiationReady* increases by 0.0242 (95% CI $[0.0060, 0.0423]$, $p = 0.0053$), while *StrictInstantiationReady* increases by 0.0302 (95% CI $[0.0060, 0.0544]$, $p = 0.0142$; Table 7). These results provide evidence that the downstream retrieval-related improvements, the modest oracle gain, and the small but real ablation gain are unlikely to be explained by sampling variation.

To test whether strong lexical retrieval depends on matching specific numeric values between the query and the schema text, we re-ran TF–IDF, BM25, and LSA retrieval after stripping numeric tokens and/or stopwords from the query. Table 8 shows that retrieval performance is preserved or even improved after sanitization. This indicates that numeric-token overlap is not necessary for the observed retrieval performance, and that stopword and numeric sanitization did not destroy retrieval.

However, this robustness to sanitization does not imply broad semantic generalization beyond the fixed catalog; rather, the results are consistent with retrieval relying substantially on domain-specific terms (parameter names, units, and optimization keywords) rather than on numeric literals alone. The broader external generalization of schema retrieval remains untested. The sanitization diagnostic uses a closely matched diagnostic retrieval configuration with the same 256-dimensional LSA setting. Its LSA baseline row reproduces the Table 3 value exactly (0.8459), while its TF–IDF and BM25 baseline values differ slightly from the canonical Table 3 evaluation because the diagnostic script fits the retrieval index on a 331-document catalog (excluding 4 open-domain filler templates present in the final 335-document catalog) and excludes the `doc_id` tokens from the indexed document representations, slightly shifting TF–IDF term weights. Both configurations yield nearly identical performance and lead to the same qualitative conclusion. A complementary stratification by query–schema lexical (Jaccard) overlap shows most original queries (327 of 331) fall in a high-overlap bucket with TF–IDF *Schema R@1* = 0.9174; the remaining medium-overlap bucket has only 4 queries (TF–IDF *Schema R@1* = 0.0000 on this bucket) and is too small to support a separate quantitative claim, so we report it only as a qualitative caveat on the generality of the overlap analysis.

Finally, because the paper's central interpretation depends on comparing non-oracle retrieval-conditioned grounding against the Oracle control, we address directly a natural reviewer concern: could an incorrect predicted schema still satisfy the *InstantiationReady* threshold (Eq. (8)) merely because a wrong schema happens to induce a smaller, more easily satisfied eligible-slot set, inflating non-oracle scores relative to what schema-conditioned grounding actually achieves? This concern is not hypothetical: in a repository diagnostic that evaluated a selective schema-reranking variant of the grounding stage as a candidate improvement, ordinary *InstantiationReady* rose from 257/331 to 265/331, but 6 of the 8 newly ready queries had done so under an *incorrect* predicted schema, exposing exactly this weakness in the ordinary metric rather than a genuine grounding improvement (that reranking variant is not otherwise adopted in this paper and is reported here only as the diagnostic finding that motivated the metric below). Recall that *InstantiationReady* does not include exact schema match as an explicit hard gate. To guard against this failure mode in the main results reported above, we define a stricter diagnostic variant that adds this gate,

$$\mathit{StrictInstantiationReady}_i = \mathbb{1}[\mathit{SchemaHit}_i \wedge \mathit{Coverage}_i \geq 0.8 \wedge \mathit{TypeMatch}_i \geq 0.8], \tag{9}$$

and report it as a robustness check, not as a replacement for the canonical metric. This strict gate is diagnostic of query-level schema agreement and does not alter the underlying aggregate Coverage or TypeMatch rates themselves, which continue to reflect the alignment of predicted slot inventories. Table 9 compares both criteria on the final evaluation records. Adding the schema-match gate removes 10 of 331 TF–IDF queries from the ready count in both the baseline and ratio-aware extraction configurations – cases where the predicted schema was wrong but the resulting eligible-slot set still happened to satisfy the coverage and type thresholds – while leaving Oracle unchanged by construction (Oracle always has a correct schema, so the added

Table 7 Paired bootstrap significance tests ($B = 10{,}000$ resamples, original queries) computed from the final per-query evaluation records. Diff is method A minus method B on the stated metric.

Comparison (metric)	Diff	95% CI	p
TFIDF-TG (ratio-aware) vs. Oracle-TG (InstReady)	-0.0483	$[-0.0755, -0.0242]$	< 0.001
TFIDF-TG (ratio-aware) vs. Oracle-TG (Strict)	-0.0785	$[-0.1088, -0.0514]$	< 0.001
TFIDF-TG baseline vs. ratio-aware (Strict)	$+0.0242$	$[0.0091, 0.0423]$	0.0006
BGE-M3 vs. TFIDF-TG (ratio-aware) (InstReady)	$+0.0242$	$[0.0060, 0.0423]$	0.0053
BGE-M3 vs. TFIDF-TG (ratio-aware) (Strict)	$+0.0302$	$[0.0060, 0.0544]$	0.0142

The baseline versus ratio-aware extraction comparison (8 strict gains, 0 losses) is also significant under an exact McNemar test ($p = 0.008$). CI = confidence interval. All five rows are reproducible from a single committed, deterministic script (paired percentile bootstrap, $B = 10{,}000$, seed 42, two-sided $p = 2\min(P(\Delta \leq 0), P(\Delta > 0))$ over the bootstrap resamples) applied to the frozen per-query records; see Code availability.

Table 8 Retrieval accuracy (*Schema R@1*, original queries) after removing numeric tokens and/or stopwords, using the diagnostic sanitization configuration with 256-dimensional LSA. The TF–IDF and BM25 baseline rows differ slightly from the canonical Table 3 evaluation because of the catalog and indexed-document configuration differences described in the text; the LSA baseline reproduces the Table 3 value exactly.

Sanitization	TF–IDF	BM25	LSA
None (baseline)	0.9063	0.8852	0.8459
Numbers removed	0.9063	0.8973	0.8459
Stopwords removed	0.9124	0.9063	0.9184
Numbers & stopwords removed	0.9124	0.9063	0.9184

gate is a no-op for it). The TF–IDF-vs-Oracle gap therefore *widens* under the stricter criterion (from 0.0483 to 0.0785) and remains statistically significant ($p < 0.001$ under the paired bootstrap). This shows that the qualitative conclusion remains robust under the explicit schema-match gate, and the measured TF–IDF-to-Oracle readiness gap increases under the stricter criterion.

4.6 Computational Complexity and Runtime

The deterministic components of the pipeline are lightweight by design. TF–IDF retrieval requires fitting a sparse vectorizer once over the 335-document catalog and, per query, a sparse dot product against all catalog documents, giving a per-query retrieval cost of $O(\text{nnz}(q)+M)$ where $\text{nnz}(q)$ is the number of nonzero query terms and $M = 335$ is the catalog size. Rule-based numeric extraction scans the query text once with a fixed set of regular-expression patterns, giving a cost linear in query length. Typed-greedy assignment (Eq. (4)) is $O(|\mathcal{P}| \cdot |\mathcal{T}|)$ in the number of eligible slots and extracted mentions, both of which are small (typically single digits to low tens) for

Table 9 Sensitivity check: canonical *InstantiationReady* versus the stricter *StrictInstantiationReady* (Eq. (9)), original queries, computed from the final per-query evaluation records. Δ is standard minus strict; n_{differ} counts queries whose readiness label changes.

Method	InstReady	StrictInstReady	Δ	n_{differ}
TFIDF-TG (baseline)	0.7764	0.7462	0.0302	10
TFIDF-TG (ratio-aware)	0.8006	0.7704	0.0302	10
Oracle-TG	0.8489	0.8489	0.0000	0

NLP4LP queries. None of these three stages involves model training, fine-tuning, or iterative optimization.

We measured wall-clock time and peak memory for the full frozen benchmark run (TF–IDF retrieval with ratio-aware typed-greedy grounding, 331 queries, single CPU core, `PYTHONHASHSEED=0` for determinism). The complete run, including retrieval, extraction, typing, and assignment for all queries, took 1.09 total seconds (3.29 milliseconds per query on average) with a peak resident memory of approximately 198 MB. This measurement covers only the deterministic TF–IDF-based configuration; it excludes BGE-M3 embedding computation (which requires loading a pretrained neural encoder and, in our setup, GPU acceleration, as described in Section 4.1) and excludes the external LLM baselines of Section 4.8, which depend on remote API latency rather than local computation. We do not compare runtime across methods that were not measured under comparable conditions; the figure above is reported only to substantiate the paper's claim that the proposed deterministic grounding backbone is lightweight and CPU-compatible, not as a cross-method efficiency benchmark.

4.7 Structural and Solver-Backed Validation

The benchmark above is deliberately solver-free: no feasibility test or objective evaluation enters *Schema R@1*, *Coverage*, *TypeMatch*, or *InstantiationReady*. To assess whether the recovered schema-conditioned structure has downstream value, we report three restricted downstream validation studies on subsets of NLP4LP for which reference solver implementations (`optimus_code`) are available: two static structural analyses (60 and 269 instances) and one actual solver-backed execution study (20 instances). The structural studies require no solver execution, and their reported structural metrics do not require Gurobi. For the 20-instance execution study, we use SciPy HiGHS through a compatibility shim, so Gurobi is not required to reproduce those results either. All three studies are restricted in scope by construction and are not claims of benchmark-wide executability.

We define the validation metrics once here as per-instance Boolean conditions, each reported as a rate over the relevant subset. *Structural validity* holds when the instantiated schema forms a well-formed linear or mixed-integer program with a defined objective and at least one decision variable, checked statically without invoking a solver. *Instantiation completeness* holds when every scalar slot the gold schema marks as required receives a type-consistent assignment. *Executable* holds when the generated

Table 10 Structural subset (60 instances): schema-hit, structural-validity, and instantiation-completeness rates.

Baseline	Schema Hit	Structural Valid	Inst. Complete
TF–IDF	0.9333	0.7500	0.7500
BM25	0.9000	0.7333	0.7333
Oracle	1.0000	0.7667	0.7833

solver program runs to completion without raising an exception. *Solver success* holds when the solver returns a well-defined termination status rather than erroring. *Feasible* holds when the solver reports a feasible solution, and *objective produced* holds when it returns a finite objective value. Structural validity and instantiation completeness are static checks; the remaining four require actually executing the solver.

Structural subset (60 instances)

Table 10 reports schema-hit, structural-validity, and instantiation-completeness rates on a 60-instance subset. TF–IDF reaches a structural-valid rate of 0.75 and an instantiation-complete rate of 0.75; Oracle retrieval reaches 0.7667 and 0.7833, respectively. As in the main benchmark, the oracle gain over TF–IDF is small, showing that the same qualitative pattern is consistent with the main benchmark: schema selection provides only modest additional gains, while substantial downstream limitations remain even on this smaller subset.

Extended structural subset (269 instances)

A larger 269-instance subset with available reference solver code was also examined to corroborate structural behavior. On this extended subset, TF–IDF, BM25, and Oracle achieved schema-hit rates of 0.9368, 0.9257, and 1.0000, structural-validity rates of 0.8141, 0.8104, and 0.8253, and instantiation-completeness rates of 0.6654, 0.6580, and 0.6840, respectively. Solver execution was not evaluated for this subset. At the time of the reported evaluation, the reference implementations required Gurobi functionality that was unavailable in the evaluation environment; the reported results for this subset are therefore structural only. We therefore treat this subset as structural evidence only and reserve executable conclusions for the HiGHS-compatible subset below.

Restricted solver-backed subset (20 instances)

To obtain direct solver-backed evidence, we further filter to a 20-instance subset selected deterministically by a static, outcome-independent compatibility rule. Specifically, we load the canonical test split (preserving its default ordering) and select the first 20 instances where the gold `optimus_code` does not contain unsupported commercial solver constructs (such as piecewise linear objectives or specialized API constraints) and is thus compatible with our open-source SciPy HiGHS compatibility shim. Crucially, the subset was fixed before evaluating either retrieval condition and is identical across baselines (the same 20 IDs are used for both TF–IDF and Oracle), and it is independent of any baseline prediction or solver outcome. Table 11 reports

Table 11 Restricted solver-backed subset (20 instances, SciPy HiGHS MILP compatibility shim; Gurobi not required).

Baseline	Executable	Solver Success	Feasible	Objective Produced
TF–IDF	0.95	0.80	0.80	0.80
Oracle	0.95	0.75	0.75	0.75

executable, solver-success, feasible, and objective-produced rates for TF–IDF and Oracle retrieval. TF–IDF reaches 0.95 executable, 0.80 solver-success, 0.80 feasible, and 0.80 objective-produced; Oracle reaches 0.95, 0.75, 0.75, and 0.75, respectively. The non-executable cases in each baseline fail due to an indexing incompatibility in the shim path (`IndexError`), affecting 2 of the 40 baseline–instance evaluations across the two baselines. Because the subset is very small ($n = 20$) and selected via compatibility filtering rather than random sampling, the small difference in rates (16/20 or 0.80 for TF–IDF vs. 15/20 or 0.75 for Oracle on solver-success, feasibility, and objective-produced rates) is statistically unstable and inconclusive, rather than evidence that TF–IDF genuinely outperforms Oracle. This subset should be read solely as a restricted, pragmatic demonstration that solver execution is indeed possible on compatible cases, not as a benchmark-wide claim of solver readiness.

Summary

Across all three validation studies, the same qualitative pattern from the main benchmark repeats: Oracle retrieval does not consistently provide a substantial advantage over TF–IDF, while substantial downstream limitations remain. On the structural subset Oracle is modestly higher than TF–IDF (0.7667 vs. 0.75 structural-valid; 0.7833 vs. 0.75 instantiation-complete), but on the restricted solver-backed subset Oracle is actually *lower* than TF–IDF on solver-success, feasible, and objective-produced rates (0.75 vs. 0.80 on each), a difference that is statistically unstable and inconclusive due to the small size ($n = 20$) and compatibility filtering of the subset, rather than a genuine performance effect. Taken together, these restricted studies show that the same qualitative pattern is consistent with the main benchmark: schema selection provides only modest additional gains, while substantial downstream limitations remain, and they caution against assuming a monotonic Oracle advantage on small subsets. In the extended structural analysis, missing scalar slots, type mismatch, and incomplete instantiation account for the structural gaps (69, 82, and 267 counted instances, respectively, across baselines), consistent with the error taxonomy in Section 4.4. These restricted studies should be read as evidence of downstream engineering relevance on compatible cases, complementing – not replacing – the main scalar-grounding benchmark.

4.8 External Baseline Comparison on a Shared Subset

The proposed method performs fixed-catalog scalar grounding and does not generate complete formulations, solver code, or executable models. Most external systems rely on generative LLM inference to synthesize full formulations or code, whereas

our evaluated grounding stage is deterministic and non-generative; consequently, the shared subset is intended only as contextual execution evidence. Its native metrics (Section 4.1) are *not numerically comparable* to the end-to-end outcomes reported by LLM-based modeling systems, and any direct head-to-head ranking would be misleading. To nonetheless provide context, we report a contextual external-baseline assessment on a small shared subset of NLP4LP-derived problems for which several recent LLM systems were run under a common execution harness. Table 12 summarizes the implementation provenance and evaluation context of each system; Table 13 records only the shared end-to-end outcome cells that were actually evaluated, with unavailable cells shown as "not applicable" rather than as zero. The complete protocol and per-instance evaluation artifacts are available in the accompanying public repository.

These results must be read with explicit fairness caveats, and they are *not* a leaderboard. First, the systems solve different tasks: the proposed method performs fixed-catalog scalar grounding, whereas PaMOP, ORLM, OptMATH, and the generic LLM generate complete formulations or solver code; for this reason the proposed method is excluded from every shared end-to-end cell in Table 13. Second, evaluation provenance differs: PaMOP was independently reconstructed using Azure OpenAI model snapshot `gpt-5.4-2026-03-05`, evaluated on 2026-08-15, with temperature 0.2, rather than the paper's original configuration. Its objective agreement is calculated within a 5% relative tolerance on execution-successful and evaluable cases, not a structural or semantic correctness judgment. Third, ORLM's execution cells are "not applicable" rather than zero because its solver dependency (`coptpy`) is unavailable in the evaluation environment. OR-R1 and DeepOR are excluded from Table 13 for a stronger reason than a scheduling or integration choice: DeepOR has neither official code nor a released checkpoint that we could locate, and OR-R1's official code (`SCUTE-ZZ/OR-R1` on GitHub) is public but no trained checkpoint has been released for it, so no evaluable artifact exists for either system and no empirical row can be constructed. Fourth, the generic LLM is a general-purpose model evaluated directly under the common harness rather than a reproduction of a published optimization-specific method; its objective-agreement rate relative to OptMATH must be read with the temperature and training caveats in mind (OptMATH temperature 0.8 with its official checkpoint `Aurora-Gem/OptMATH-Qwen2.5-7B`; generic `gpt-5.4-2026-03-05` snapshot evaluated on 2026-08-15 with temperature 0.0), never as a ranking of optimization-modeling systems. Finally, the shared subset is small (18 instances) and selection was outcome-independent, but it remains a feasibility-oriented probe rather than a benchmark-scale comparison. We therefore treat this assessment as supplementary context for the qualitative positioning in Section 2.4, not as evidence about which system is "better."

4.9 External Component-Level Robustness Validation

As an independent component-level robustness check, we evaluated the deterministic numeric-extraction rules on a fixed 1,000-instance sample of the OptMATH-Train dataset [19]. Because OptMATH does not provide the fixed schema catalog required by our main task, this experiment evaluates numeric extraction only, rather than

schema retrieval or schema-conditioned grounding. Under an automated classification baseline, the raw AST-based code-value overlap recall is 0.7501, and the initial automatic text-support estimate of recall is 0.8118 (with a 0.5910 complete instance extraction rate, defined as the fraction of instances for which all automatically classified text-supported numeric values were recovered).

However, a deterministic secondary rule-based classifier pass over a random sample of 150 instances (seed 20260815) reveals significant limitations in using AST-derived code overlap as an end-to-end modeling metric. We find that 7.28% of the original AST denominator (computed over the full 1,000-instance sample) consists of implementation, loop, or solver-specific constants (such as array sizes or Gurobi parameters) that do not express semantic numeric parameters in the problem text.

After auditing code-derived constants and restricting evaluation to classifier-identified text-supported values, the audit-calibrated text-supported recall is 0.8070 (95% bootstrap confidence interval $[0.7842, 0.8283]$, denominator $n = 28{,}799$, which represents automatically classified text-supported values across the 1,000-instance sample rather than manually reviewed literals). This denominator was derived by running an automatic text-support classifier over the 1,000-instance sample to exclude loop bounds, index variables, and implementation constants. The accuracy of this primary classifier was cross-checked against a second, independently coded, deterministic rule-based classifier applied to a 150-instance random sample (covering 4,801 literals), giving a 98.04% agreement rate between the two automated classifiers (94 disagreements out of 4,801 literals). This is an automated cross-validation between two rule-based classifiers, not a human-annotated gold-label audit; no manual, human-reviewed annotation of these literals was performed, so the calibration should be read as agreement between two independently implemented heuristics rather than as validation against ground truth. Overall, this assessment provides limited evidence that the deterministic numeric-extraction component transfers beyond NLP4LP; it does not, however, establish external generalization of the complete framework or any grounding/retrieval module.

5 Conclusions and Future Research

This paper studied a retrieval-assisted approach to optimization problem instantiation from natural-language descriptions. Rather than targeting full generative model synthesis, we focused on a narrower intermediate task: retrieving a compatible optimization schema from a fixed catalog and then instantiating its scalar parameters through deterministic grounding of numeric mentions in the query. This formulation isolates an important intermediate capability in natural-language-to-optimization pipelines: before full model synthesis can be attempted, the system must identify a plausible template and recover at least part of the quantitative information needed to instantiate it.

The benchmark results support several clear conclusions. First, classical lexical retrieval (e.g., TF–IDF) is already strong for fixed-catalog schema identification. Second, modern pretrained dense retrieval (BGE-M3) improves schema identification

significantly, and this improvement translates into higher downstream readiness metrics. Third, despite these improvements, a residual gap to oracle schema selection remains; perfect schema selection yields only a modest additional gain in downstream readiness over the strongest evaluated retrieval setting. Consequently, downstream scalar grounding and parameter recovery remain important unresolved components in this fixed-catalog benchmark.

The proposed grounding procedure remains deterministic and does not require generative LLM inference. While this property ensures that intermediate assignments are fully inspectable, the numeric-extraction ablation and error taxonomy indicate that the deterministic extraction, typing, and assignment rules remain insufficient to resolve some semantic ambiguities.

Beyond the core benchmark, the restricted structural and solver-backed validation studies suggest downstream optimization-oriented usefulness on compatible cases, though they do not establish benchmark-wide solver readiness.

5.1 Limitations and Threats to Validity

The scope restrictions adopted throughout this paper (Section 3.1) are deliberate design choices, but they also bound what the results can and cannot support. We summarize the main limitations and threats to validity here rather than leaving them dispersed across the paper.

Fixed-catalog retrieval and task scope. Schema retrieval is evaluated only as a benchmark-specific 335-way identification problem over the catalog described in Section 3.2. These strong retrieval numbers characterize this fixed-catalog setting and must not be interpreted as implying that open-domain schema retrieval is solved. Rather, our focus is explicitly on studying schema-conditioned scalar grounding as a distinct subproblem, rather than claiming full, unconstrained model generation. Generalization to substantially larger or open-domain schema libraries remains untested even with the dense retrieval baseline.

Dataset dependence and lexical overlap. All quantitative results are computed on NLP4LP, a single benchmark. The overlap-based stress tests in Section 4.5 show that retrieval accuracy is not simply an artifact of numeric-value overlap between query and schema text, but the diagnostic medium-overlap bucket contains only 4 of 331 original queries – too few to fully rule out benchmark-specific lexical or structural regularities at scale. Generalization to optimization problem descriptions written in different styles, domains, or languages is untested.

Gated data access. NLP4LP requires an approved Hugging Face account and access token (Declarations, Data availability). Independent researchers can inspect and rerun analyses on the committed result artifacts without this access, but a full end-to-end rerun on live benchmark queries requires it, which is a real (if benchmark-typical) reproducibility constraint.

Scalar-only instantiation. The benchmarked instantiation stage recovers only scalar parameters; list-valued, vector-valued, and dictionary-valued parameters are out of scope (Section 3.4). The reported *Coverage*, *TypeMatch*, and *Instantiation-Ready* scores therefore characterize a restricted parameter-recovery problem, not full model-parameter reconstruction.

Heuristic type and role inference. Both coarse type inference (Section 3.3) and optional role-matching cues (Section 3.3) are deterministic, lexicon- and rule-based mechanisms with no formal correctness guarantee. The residual-error decomposition (Table 5) indicates that this heuristic layer remains an important unresolved component of downstream scalar recovery, and that fine-grained value-to-slot ambiguity specifically – rather than coarse type identification – is not resolved by the methods evaluated here.

Restricted and small solver-backed validation. The structural and solver-backed studies in Section 4.7 use restricted subsets (60, 269, and 20 instances, respectively) selected by data-availability or compatibility filtering rather than random sampling, and the proposed pipeline's solver-backed validation in Section 4.7 uses only SciPy's open-source HiGHS backend; Gurobi and other commercial or specialized solvers are not exercised. The 20-instance restricted solver-backed subset in particular is too small to support benchmark-wide solver-readiness claims, and the larger 269-instance subset was used only for structural analysis because its reference implementation depended on Gurobi support that was unavailable in the evaluation environment at the time of evaluation.

Limited direct comparison against end-to-end LLM systems. Section 2 discusses several recent end-to-end large-language-model-based optimization-modeling systems (e.g., OptiMUS, OptLLM, Chain-of-Experts, LLMOPT, OPT2CODE, ORLM, OptMATH) qualitatively, and the repository documents a contextual external-baseline assessment on a small common-18 subset in which PaMOP (an independent reconstruction rather than an official reproduction) completes 18/18 model-generation attempts (with 13 of 18 yielding executable AMPL files), OptMATH achieves 18/18 generation and 15/18 solver execution, and a generic LLM (acting as a generic zero-shot reference rather than an optimization-trained baseline) completes 16/18, while ORLM's released checkpoint execution is not applicable (N/A) because its solver dependency (`coptpy`) is unavailable, and DeepOR and OR-R1 have no evaluable released artifact (DeepOR has neither official code nor a checkpoint; OR-R1's official code is public but its checkpoint was never released), so neither could be evaluated in our common harness regardless of integration effort. However, this paper does not run these systems head-to-head against the proposed pipeline on the full NLP4LP benchmark. These systems target a different, broader task (full formulation and solver-code generation); a full-scale controlled evaluation of these end-to-end systems across the complete NLP4LP benchmark would require substantially greater model-access and computational resources and was outside the scope of this study. A controlled quantitative comparison is therefore left to future work.

No external end-to-end framework validation or deployment study. Section 4.8 provides external component-level validation of numeric extraction on OptMATH, but schema retrieval and schema-conditioned grounding remain evaluated only on NLP4LP. Consequently, cross-benchmark generalization of the complete framework remains untested. Similarly, no user study or production deployment was conducted; the practical-usefulness claims in this paper are limited to the restricted, benchmark-scoped, and subset-based evidence reported in Sections 4.3–4.7.

5.2 Future Research Directions

Several directions for future research follow naturally from these findings. The first is to strengthen the downstream grounding stage itself with substantially stronger semantic role-sensitive methods, rather than additional variants of the current deterministic assignment family. The second is to extend the current scalar-only setting to more structured parameter types, including vectors, indexed collections, and dictionary-valued arguments. The third is to explore hybrid systems in which schema retrieval provides structural scaffolding for stronger learned or LLM-based grounding modules. A fourth direction is to expand the current restricted engineering validations into broader executable studies with more complete solver-backed coverage and wider template support.

More broadly, the results suggest that intermediate capabilities such as schema retrieval, schema-conditioned slot selection, and partial structured instantiation deserve independent attention in the design of transparent optimization support systems. Full natural-language-to-optimization automation remains difficult, but intermediate systems can already provide useful support by narrowing candidate problem structure and recovering part of the quantitative content needed for formal modeling. At the same time, the present results make clear that robust downstream scalar recovery, including quantity-to-role grounding, remains unresolved even when retrieval is strong. This makes the intermediate task studied here both practically relevant and scientifically valuable as a benchmarked subproblem in natural-language-to-optimization research.

Declarations

Funding. This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. The external large-language-model baselines reported in Section 4.8 used Microsoft Azure OpenAI API access.

Competing interests. The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate. Not applicable; this study did not involve human participants, human data, or animals.

Author contributions. Soroush Vahidi: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing – original draft, Writing – review & editing, Visualization. This single-author paper uses the standard CRediT taxonomy to describe the contributions of the sole author.

Data availability. The benchmark used in this paper, NLP4LP (`udell-lab/NLP4LP` on Hugging Face), is a gated third-party dataset. The author does not redistribute this dataset; eligible researchers must obtain it directly from its original Hugging Face source under its specific access conditions. The derived artifacts supporting the results reported in this paper, together with the corresponding tables, figures, and analysis outputs, are publicly available in the accompanying GitHub repository at https://github.com/SoroushVahidi/combinatorial-opt-agent.

Code availability. The retrieval, grounding, evaluation, and analysis code supporting this paper is available at https://github.com/SoroushVahidi/combinatorial-opt-agent. Reproducing a full NLP4LP rerun requires the gated dataset access described above.

Declaration of generative AI and AI-assisted technologies in the writing process. During the preparation of this work the author used ChatGPT and Gemini to improve readability and language. After using these tools, the author reviewed and edited the content as needed and takes full responsibility for the content of the published article.

Declaration of AI assistance in software development. During the research and development process, the author used Cursor and GitHub Copilot for coding assistance; the author reviewed, verified, and edited all code and takes full responsibility for the implementation. Note that the scalar grounding procedure evaluated in this paper is deterministic at inference time and does not itself rely on generative LLM inference; schema retrieval is evaluated using both deterministic lexical methods and the pretrained BGE-M3 dense encoder (with the exception of the external baselines evaluated separately in Section 4.8).

References

[1] Affolter K, Stockinger K, Bernstein A (2019) A comparative survey of recent natural language interfaces for databases. The VLDB Journal 28(5):793–819. https://doi.org/10.1007/s00778-019-00567-8

[2] AhmadiTeshnizi A, Gao W, Udell M (2023) NLP4LP: A dataset for natural language to linear programming and mixed-integer linear programming. Hugging Face dataset repository, URL https://huggingface.co/datasets/udell-lab/NLP4LP, available at the udell-lab/NLP4LP dataset page

[3] AhmadiTeshnizi A, Gao W, Udell M (2024) OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In: Proceedings of the 41st International Conference on Machine Learning, Proceedings of Machine Learning Research, vol 235. PMLR, pp 577–596, URL https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html

[4] Ahmed T, Choudhury S (2025) OPT2CODE: A retrieval-augmented framework for solving linear programming problems. Natural Language Processing Journal 13:100185. https://doi.org/10.1016/j.nlp.2025.100185, URL https://www.sciencedirect.com/science/article/pii/S2949719125000615

[5] Astorga N, Liu T, Xiao Y, et al (2025) Autoformulation of mathematical optimization models using LLMs. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning Research, vol 267. PMLR, pp 1864–1886, URL https://proceedings.mlr.press/v267/astorga25a.html

[6] Atzeni P, Baldazzi T, Bellomarini L, et al (2026) Semantic-aware query answering with large language models. Data & Knowledge Engineering 161:102494. https://doi.org/10.1016/j.datak.2025.102494

[7] Biemans B, Troubil P, Grau I, et al (2026) Explainable optimization: Leveraging large language models for user-friendly explanations. In: Guidotti R, Schmid U, Longo L (eds) Explainable Artificial Intelligence, Communications in Computer and Information Science, vol 2578. Springer, Cham, p 44–67, https://doi.org/10.1007/978-3-032-08327-2_3, URL https://link.springer.com/chapter/10.1007/978-3-032-08327-2_3, xAI 2025, first online 12 October 2025

[8] Chen J, Xiao S, Zhang P, et al (2024) M3-Embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In: Findings of the Association for Computational Linguistics: ACL 2024. Association for Computational Linguistics, pp 2318–2335, https://doi.org/10.18653/v1/2024.findings-acl.137

[9] Chen Y, Xia J, Shao S, et al (2025) Solver-informed RL: Grounding large language models for authentic optimization modeling. In: Advances in Neural Information Processing Systems (NeurIPS)

[10] Deerwester S, Dumais ST, Furnas GW, et al (1990) Indexing by latent semantic analysis. Journal of the American Society for Information Science 41(6):391–407. https://doi.org/10.1002/(SICI)1097-4571(199009)41:6<391::AID-ASI1>3.0.CO;2-9

[11] Ding Z, Tan Z, Zhang J, et al (2026) OR-R1: Automating modeling and solving of operations research optimization problem via test-time reinforcement learning. In: Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26), pp 228–236, https://doi.org/10.1609/aaai.v40i1.36983

[12] Gao Z, Ge A, Berenbeim A, et al (2026) Models can model, but can't bind: Structured grounding in text-to-optimization. In: Conference on Language Modeling (COLM)

[13] Huang C, Tang Z, Hu S, et al (2025) ORLM: A customizable framework in training large models for automated optimization modeling. Operations Research 73(6):2986–3009. https://doi.org/10.1287/opre.2024.1233

[14] Jiang C, Shu X, Qian H, et al (2025) LLMOPT: Learning to define and solve general optimization problems from scratch. In: Proceedings of the Thirteenth International Conference on Learning Representations, URL https://openreview.net/forum?id=9OMvtboTJg

[15] Kadioglu S, Dakle PP, Uppuluri K, et al (2024) Ner4Opt: Named entity recognition for optimization modelling from natural language. Constraints 29(3–4):261–299. https://doi.org/10.1007/s10601-024-09376-5

[16] Lhoest Q, Villanova del Moral A, Jernite Y, et al (2021) Datasets: A community library for natural language processing. In: Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, pp 175–184, https://doi.org/10.18653/v1/2021.emnlp-demo.21, URL https://aclanthology.org/2021.emnlp-demo.21/

[17] Li Y, Wang H, Xue B, et al (2026) Solver-independent automated problem formulation via LLMs for high-cost simulation-driven design. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.findings-acl.102

[18] Li Z, Lu H, Wei T, et al (2026) Constructing industrial-scale optimization modeling benchmark. In: Proceedings of the 43rd International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, vol 306. PMLR

[19] Lu H, Xie Z, Wu Y, et al (2025) OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In: Proceedings of the 42nd International Conference on Machine Learning (ICML 2025), Proceedings of Machine Learning Research, vol 267. PMLR, pp 40769–40802, URL https://proceedings.mlr.press/v267/lu25o.html

[20] Pan X, Fang J, Wu F, et al (2025) Guiding large language models in modeling optimization problems via question partitioning. In: Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence (IJCAI-25), pp 2657–2665, https://doi.org/10.24963/ijcai.2025/296, URL https://www.ijcai.org/proceedings/2025/296

[21] Pedregosa F, Varoquaux G, Gramfort A, et al (2011) Scikit-learn: Machine learning in Python. Journal of Machine Learning Research 12:2825–2830

[22] Powell C, Riccardi A (2025) Generating textual explanations for scheduling systems leveraging the reasoning capabilities of large language models. Journal of Intelligent Information Systems 63(4):1287–1337. https://doi.org/10.1007/s10844-025-00940-w, URL https://doi.org/10.1007/s10844-025-00940-w

[23] Ramamonjison R, Li H, Yu TTL, et al (2022) Augmenting operations research with auto-formulation of optimization models from problem descriptions. In: Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing: Industry Track. Association for Computational Linguistics, pp 29–62, https://doi.org/10.18653/v1/2022.emnlp-industry.4, URL https://aclanthology.org/2022.emnlp-industry.4/

[24] Ramamonjison R, Yu T, Li R, et al (2023) NL4Opt competition: Formulating optimization problems based on their natural language descriptions. In: Proceedings of the NeurIPS 2022 Competitions Track, Proceedings of Machine Learning Research, vol 220. PMLR, pp 189–203, URL https://proceedings.mlr.press/v220/ramamonjison23a.html

[25] Rizzi S, Francia M, Gallinucci E, et al (2025) Conceptual design of multidimensional cubes with LLMs: An investigation. Data & Knowledge Engineering 159:102452. https://doi.org/10.1016/j.datak.2025.102452

[26] Robertson S, Zaragoza H (2009) The probabilistic relevance framework: BM25 and beyond. Foundations and Trends in Information Retrieval 3(4):333–389. https://doi.org/10.1561/1500000019

[27] Sheetrit E, Brief M, Mishaeli M, et al (2024) ReMatch: Retrieval enhanced schema matching with LLMs. arXiv preprint arXiv:240301567 https://doi.org/10.48550/arXiv.2403.01567, URL https://arxiv.org/abs/2403.01567

[28] Xiao Z, Zhang D, Wu Y, et al (2024) Chain-of-experts: When LLMs meet complex operations research problems. In: Proceedings of the Twelfth International Conference on Learning Representations, URL https://openreview.net/forum?id=HobyL1B9CZ

[29] Xiao Z, Wang YJ, Han X, et al (2026) DeepOR: A deep reasoning foundation model for optimization modeling. In: Proceedings of the Fortieth AAAI Conference on Artificial Intelligence (AAAI-26), pp 34052–34060, https://doi.org/10.1609/aaai.v40i40.40699

[30] Xie W, Chen Y, Hu YX, et al (2026) Escaping the sisyphus dilemma: Experience replay for robust text-to-optimization modeling. In: Findings of the Association for Computational Linguistics: ACL 2026. Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.findings-acl.690

[31] Yu L, Zhao X, Li D, et al (2026) Uncertainty-aware test-time search for optimization problem solving. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.acl-long.1975

[32] Yu T, Zhang R, Yang K, et al (2018) Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing. Association for Computational Linguistics, pp 3911–3921, https://doi.org/10.18653/v1/D18-1425, URL https://aclanthology.org/D18-1425/

[33] Zhai H, Lawless C, Vitercik E, et al (2025) EquivaMap: Leveraging LLMs for automatic equivalence checking of optimization formulations. In: Proceedings of the 42nd International Conference on Machine Learning (ICML), Proceedings of Machine Learning Research, vol 267. PMLR, pp 74288–74305

[34] Zhang J, Wang W, Guo S, et al (2024) Solving general natural-language-description optimization problems with large language models. In: Proceedings of the 2024 Conference of the North American Chapter of the Association for

Computational Linguistics: Human Language Technologies (Volume 6: Industry Track). Association for Computational Linguistics, pp 483–490, https://doi.org/10.18653/v1/2024.naacl-industry.42, URL https://aclanthology.org/2024.naacl-industry.42/

[35] Zhang X, Wan Y, Zhang B, et al (2026) Dual-cluster memory agent: Resolving multi-paradigm ambiguity in optimization problem solving. In: Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, https://doi.org/10.18653/v1/2026.acl-long.266

Table 12 Context and implementation provenance of the systems in the external-baseline assessment. "Cases" is the number of common-subset instances processed or evaluated under the system's native protocol. For the proposed method, Cases = 18 indicates that the same common-subset instances were processed under its native scalar-grounding protocol; because its output type is not comparable to full formulation/code generation, no shared end-to-end outcome cells are reported in Table 13. Provenance levels: NATIVE (proposed pipeline), INDEPENDENT (independent reconstruction of the published method), ADAPTED-OFFICIAL (official released checkpoint adapted to a common harness), GENERIC-ZERO-SHOT (a general-purpose model evaluated directly under the common harness rather than a reproduction of a published optimization-specific method). The proposed method is listed for context only; its native metrics are not comparable to the end-to-end cells in Table 13.

System	Provenance	Output representation	Solver environment / status	Cases
Ours (this paper)	NATIVE	Scalar slot assignments on retrieved schema	No solver call in core benchmark	18
PaMOP [20]	INDEPENDENT	AMPL formulation + solver code	Local AMPL + HiGHS; 13/18 executed	18
ORLM [13]	ADAPTED-OFFICIAL	LP formulation + solver code	Solver dependency (coptpy) unavailable; execution blocked	18
OptMATH [19]	ADAPTED-OFFICIAL	LP formulation + solver code	Gurobi (gurobipy); 15/18 executed	18
Generic LLM (GPT-5.4-2026-03-05)	GENERIC-ZERO-SHOT	Gurobi solver code, zero-shot	Gurobi (gurobipy); 16/18 executed	18
DeepOR [29]	—	Formulation (paper-described)	Neither official code nor a released checkpoint could be located; no evaluable artifact exists	0
OR-R1 [11]	—	Formulation + solver code	Official code is public, but no trained check-	0

Table 13 Shared outcomes on the 18-instance common subset, as actually evaluated by the external-baseline harness. "N/A" denotes an outcome cell that was not evaluated (never a silent zero). *Parse* is the fraction of instances whose produced artifact parsed; *Executable* is the fraction whose solver code ran to completion. These represent distinct outcome stages, though for PaMOP on this subset the observed rates happen to coincide. *Feasible* is the fraction of successfully executed runs that reported a feasible solution (not infeasible or unbounded); *Objective agreement* is the fraction of execution-successful cases with a comparable reference objective whose generated objective agrees within a 5% relative tolerance. Denominators vary across systems because they depend on how many runs reached the corresponding stage.

System	Parse	Executable	Feasible	Objective agreement
Ours (this paper)	N/A	N/A	N/A	N/A
PaMOP [20]	0.72	0.72	1.00	0.73 (8/11)
ORLM [13]	1.00	N/A	N/A	N/A
OptMATH [19]	1.00	0.83	1.00	0.40 (6/15)
Generic LLM (GPT-5.4-2026-03-05)	1.00	0.89	1.00	0.63 (10/16)